

System Design Basics

Lesson 1: Introduction to System Design

Introduction to System Design - Study Notes

Key Concepts

1. **System Design Definition** – The process of defining the architecture, components, interfaces, and data for a system to satisfy specific requirements.
2. **Design Goals** – Scalability, reliability, performance, maintainability, security, and cost effectiveness.
3. **Trade offs** – Balancing conflicting goals (e.g., consistency vs. latency) and making informed compromises.
4. **High Level Architecture** – Dividing a system into services, databases, caches, load balancers, and networking layers.
5. **Real World Patterns** – Common building blocks such as sharding, replication, caching, and asynchronous processing.

Summary

System design is the discipline of planning how a software system will be built, how its parts will interact, and how it will meet functional and non functional requirements. At the beginner level, the focus is on understanding *why* we design systems (to handle growth, avoid failures, and keep costs reasonable) and *what* the major goals are: scalability (the ability to serve more users), reliability (staying up and correct), performance (fast response times), maintainability (easy to evolve), security (protect data), and cost efficiency.

A classic introductory example is a **URL shortener** (e.g., bit.ly). Though it appears simple—a service that maps a short alias to a long URL—it must handle millions of reads per second, store billions of mappings, generate unique short keys without collisions, and stay available even if parts of the infrastructure fail. By dissecting this example, we see how concepts like database sharding, caching, load balancing, and asynchronous background jobs come together to satisfy the design goals.

Important Points

- **System design "coding"** – it's about high level decisions, not line by line implementation.
- **Non functional requirements (NFRs)** often drive the architecture more than functional specs.
- **Scalability** can be vertical (bigger machines) or horizontal (more machines); horizontal scaling is usually preferred for large scale services.
- **Reliability patterns** include replication, failover, health checks, and graceful degradation.
- **Caching** reduces load on primary data stores and improves latency; choose cache invalidation strategy wisely.
- **Data partitioning (sharding)** spreads data across multiple nodes to avoid hot spots.
- **Consistency models** range from strong (all reads see latest write) to eventual (reads may be stale temporarily).

- **Cost considerations**: over engineering can waste resources; aim for “good enough” based on expected traffic.
- **Iterative design**: start with a simple MVP, then identify bottlenecks and evolve the architecture.

Examples

1. URL Shortener

| Component | Role | Design Choice |

|-----|-----|-----|

| **API Layer** | Accepts `POST /shorten` and `GET /{code}` requests | Stateless servers behind a load balancer (horizontal scaling) |

| **Database** | Stores `code` → `longURL` mapping | NoSQL key value store (e.g., DynamoDB) with primary key = `code`; use **sharding** for high write throughput |

| **Cache** | Serves frequent redirects quickly | In memory cache (Redis/Memcached) with TTL; cache aside pattern |

| **ID Generator** | Produces unique short codes | Base 62 encoding of an auto incrementing counter (centralized) **or** distributed Snowflake style IDs to avoid a single point of failure |

| **Analytics Pipeline** | Tracks clicks for reporting | Asynchronous queue (Kafka) → worker → analytics DB; does not block redirect latency |

Why it works: The read heavy workload is offloaded to the cache; writes are spread across shards; the stateless API layer can be scaled out; failures in one component (e.g., a cache node) don't bring the whole service down.

2. Simple Chat Application (Bonus Example)

- **Goal**: Real time message delivery to millions of concurrent users.
- **Key Design Elements**: WebSocket servers behind a load balancer, message broker (e.g., RabbitMQ/Kafka) for fan out, user presence stored in a fast KV store, eventual consistency for offline message storage.
- **Illustrates**: Asynchronous processing, horizontal scaling of connection heavy services, and trade offs between latency and durability.

Quick Review

1. **What are the primary non functional goals you should consider when designing a system?**
2. **Explain the difference between vertical and horizontal scaling, and why horizontal scaling is usually preferred for large web services.**
3. **In a URL shortener, why is caching important and what caching pattern is commonly used?**
4. **What is sharding and how does it help a system handle high write volumes?**
5. **Name two reliability techniques you would apply to keep a service available during node failures.**

Further Reading

- **CAP Theorem** – Understanding the trade offs between Consistency, Availability, and Partition tolerance.
- **Design Patterns for Scalable Systems** – Caching, Rate Limiting, Circuit Breaker, Bulkhead.
- **Data Modeling for NoSQL** – When to choose key value, document, column family, or graph

stores.

- ****Load Balancing Algorithms**** – Round robin, least connections, IP hash, and health check mechanisms.

- ****Monitoring & Observability**** – Metrics, tracing, and alerting fundamentals for production systems.

Use these notes as a quick reference while you practice designing simple services. The more you break down real world examples, the easier it becomes to internalize the core principles of system design.

